

Date: 10/04/2026

Duration: 3 hours

FINAL ASSESSMENT

II YEAR – SEMESTER 4

CSE23AE204 - PCP III

(B.Tech. E01,E02,E03,E05,E06)

SET E

Question 1: Problem Statement

You are developing a **discount calculation module** for an e-commerce platform.

The platform processes **one order at a time**, provided as a **space-separated string**.

Each order belongs to a specific customer category and follows different discount rules.

The system must:

- Identify the order type from input
- Apply discounts based on customer category
- Calculate the final payable amount
- Use **inheritance and polymorphism** to support multiple order types
- Use **method overloading** to apply different discount strategies

Discount Rules

Regular Order

- A percentage-based discount is applied on the original price.
- The discounted amount is subtracted from the original price.
- The remaining value is the final payable amount.

Prime Order

- A percentage-based discount is applied on the original price.
- The discounted amount is subtracted from the original price.
- An additional fixed prime discount is further subtracted.
- The remaining value is the final payable amount.

Example 1

Input

```
{ "order": "REGULAR Ravi 2000 10" }
```

Output

1800.0

Explanation

A percentage discount is applied on the original price and subtracted to get the final payable amount.

Question 2: Problem Statement

A customer support analytics system analyzes **customer chat messages** to detect **symmetric emotional patterns**.

A message is considered **symmetric** if a **contiguous part of the message reads the same forwards and backwards** (palindrome).

Given a string message, your task is to identify and return the **longest symmetric substring** present in the message.

If multiple substrings of maximum length exist, return **any one of them**.

Example 1

Input:

message = "babad"

Output:

"bab"

Explanation:

Possible palindromic substrings are "bab" and "aba".

Both have length 3. Returning "bab" is valid.

Question 3: Problem Statement

You are given a JSON object:

```
{ "jobs": [[id, deadline, profit], ...] }
```

Each job has:

- id → Unique job identifier
- deadline → The last time slot by which the job must be completed
- profit → Profit earned if the job is completed

Rules:

- Each job takes exactly **1 unit of time**.
- Only **one job can be executed at a time**.
- A job must be completed **on or before its deadline**.

Your task is to schedule jobs to maximize total profit.

Example 1:

Input: { "jobs": [[1, 4, 20],[2, 1, 10],[3, 1, 40],[4, 1, 30]] }

Output: [2, 60]

Explanation

Step 1: Sort jobs in descending order of profit:

[3,1,40] [4,1,30] [1,4,20] [2,1,10]

Step 2: Schedule jobs in latest possible free slot:

- Job 3 → Slot 1
- Job 4 → Cannot schedule (slot 1 already filled)
- Job 1 → Slot 4
- Job 2 → Cannot schedule

Total scheduled jobs = 2

Total profit = 60

Question 4: Problem Statement

You are given:

- An array weights[] representing weight of items
- An array values[] representing value of items
- An integer capacity representing maximum weight you can carry

You can either:

- Pick an item **once**, or
- Skip it

You **cannot pick the same item multiple times**.

Your task is to find the **maximum total value** you can carry without exceeding the capacity.

Example 1

Input: weights = [1,3,4,5], values = [1,4,5,7], capacity = 7

Output: 9

Explanation:

Question 5: Problem Statement

You are given two strings s1 and s2.

You can perform the following operations on s1:

- Insert a character
- Delete a character
- Replace a character

Each operation counts as **1 step**.

Return the **minimum number of operations** required to convert s1 into s2.

Example 1

Input: s1 = "horse", s2 = "ros"

Output: 3

Explanation:

horse → rorse (replace h → r)

rorse → rose (remove r)

rose → ros (remove e)